

OE-EC803B — Big Data Analysis

Semester VIII • 3L:0T:0P • 3 Credits

Module 4: Hadoop Ecosystem Components (Pig, Hive, HBase, Big SQL)

CGEC | B.Tech ECE (2023–27) | Biraj Sarkar

Contents

1	Apache Pig	2
1.1	Execution Modes and Components	2
1.2	Pig Latin Data Model	2
1.3	Core Processing Operators	2
2	Apache Hive	3
2.1	Hive Architecture	3
2.2	Managed vs. External Tables	3
2.3	Partitioning vs. Bucketing	4
3	Apache HBase	4
3.1	HBase Architecture and Components	4
4	IBM Big SQL	5

1. Apache Pig

Apache Pig

Apache Pig is a high-level dataflow platform designed for processing large datasets on Hadoop. It provides a procedural dataflow programming language called **Pig Latin**, which compiles scripts into sequences of MapReduce jobs executed across a YARN cluster.

1.1 Execution Modes and Components

- **Local Mode:** Runs in a single JVM using the local filesystem. Excellent for testing and debugging.
- **MapReduce Mode:** Translates scripts into MapReduce jobs and executes them on an active Hadoop cluster.
- **Grunt Shell:** The interactive command-line shell used to type Pig Latin statements.
- **UDF (User Defined Functions):** Allows developers to write custom processing logic in Java, Python, or JavaScript when Pig's built-in operators are insufficient.

1.2 Pig Latin Data Model

Pig Latin supports both simple types (e.g., 'int', 'chararray') and complex nested types:

- **Atom:** Any single value stored as a string (e.g., "Biraj", '45').
- **Tuple:** An ordered set of fields (e.g., '(Biraj, 22, CGEC)').
- **Bag:** A collection of tuples (e.g., '(Biraj, 22), (Amit, 23)').
- **Map:** A set of key-value pairs (e.g., ['name' #'Biraj', 'age' #'22']).

1.3 Core Processing Operators

- **'LOAD':** Ingests data from HDFS files.
- **'FILTER':** Selects tuples from a relation based on conditions.
- **'FOREACH ... GENERATE':** Transforms data columns.
- **'GROUP':** Collects tuples sharing a key into a bag.
- **'JOIN':** Joins relations based on matching keys.

2. Apache Hive

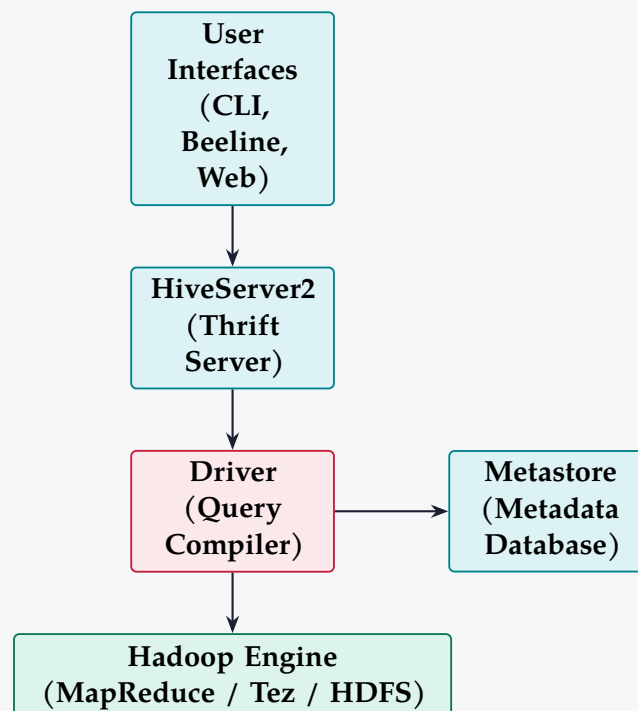
Apache Hive

Apache Hive is a data warehouse software system built on top of Apache Hadoop that provides data summarization, ad-hoc querying, and analysis of large datasets stored in HDFS. It provides an SQL-like query language called **HiveQL**.

2.1 Hive Architecture

Hive translates SQL-like HiveQL statements into MapReduce, Tez, or Spark jobs.

Hive System Architecture



- **Metastore:** A relational database (typically MySQL or Derby) that stores schema definitions, column details, and partition metadata.
- **HiveServer2:** The execution endpoint allowing remote JDBC/ODBC clients to submit queries.

2.2 Managed vs. External Tables

Hive supports two types of tables to control data ownership:

Feature	Managed Table (Internal)	External Table
Data Ownership	Hive owns both the metadata and the data.	Hive only owns the metadata.
Storage Path	Default directory: '/user/hive/warehouse/'	User-defined HDFS location.
'DROP TABLE' Impact	Deletes both the HDFS data and metadata.	Deletes metadata; HDFS data remains intact.
Use Case	Temporary tables, internal application data.	Sharing datasets with Spark, Pig, or MapReduce.

2.3 Partitioning vs. Bucketing

Hive optimizes query performance using two layout patterns:

Partitioning vs. Bucketing Properties

- **Partitioning:** Divides a table into subdirectories based on column values (e.g., '/country=IN/', '/country=US/'). It significantly reduces query search space by pruning directories.
- **Bucketing:** Distributes table rows into a fixed number of files (buckets) based on a hash function of a column (e.g., 'hash(userID) % numBuckets'). It speeds up joins and sampling on high-cardinality columns.

3. Apache HBase

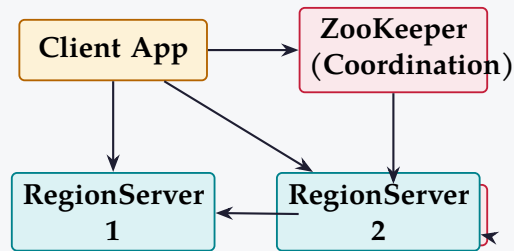
Apache HBase

Apache HBase is a distributed, column-oriented, multi-dimensional, NoSQL database built on top of HDFS, modeled after Google's Bigtable, designed to provide random, real-time read/write access to billions of rows and columns.

3.1 HBase Architecture and Components

HBase achieves horizontal scale using master-slave coordinators:

- **HMaster:** Coordinates the cluster, assigns regions to RegionServers, and handles schema updates.
- **RegionServer:** Manages a set of dynamic table splits called **Regions**. Handles client read/write requests directly.
- **Write-Ahead Log (WAL):** A local disk file where writes are written first for durability before committing to RAM.
- **MemStore:** The write buffer in RAM. Writes are accumulated here first.
- **HFile:** The physical file format stored on HDFS. When a MemStore fills up (flushes), its contents are written to disk as an HFile.

HBase Architecture

4. IBM Big SQL

IBM Big SQL

IBM Big SQL is an enterprise-grade, SQL-on-Hadoop engine that provides an ANSI-SQL compliant query compiler and Massively Parallel Processing (MPP) execution framework to query data stored across HDFS, Hive, and HBase directly.

Big SQL utilizes a shared-nothing MPP architecture that executes SQL queries natively on Hadoop data blocks without the latency overhead of MapReduce translations, supporting complex joins and standard business intelligence integrations.

Quick Revision — Module 4

Pig Latin, HiveQL, HBase NoSQL & Big SQL | OE-EC803B

One-Line Definitions

- **Pig Latin:** A high-level dataflow programming language used to describe data transformation pipelines.
- **HiveQL:** An SQL-like query language that is compiled into execution plans (like Tez or MapReduce).
- **Metastore:** The repository that stores structural metadata and schema information for Hive tables.
- **RegionServer:** An HBase slave node that serves read and write operations for a subset of row keys.
- **Big SQL:** An enterprise Massively Parallel Processing (MPP) SQL querying engine for Hadoop clusters.

Frequently Asked / Viva Questions

Q1. Compare Apache Pig and Apache Hive.

Apache Pig uses a procedural dataflow scripting language (Pig Latin) suited for ETL pipelines and data engineers. Apache Hive uses a declarative SQL-like language (HiveQL) suited for data warehousing and business intelligence.

Q2. What happens to the HDFS data when a Managed Table vs. an External Table is dropped in Hive?

Dropping a Managed Table deletes both the table metadata and the physical data files in HDFS. Dropping an External Table deletes only the Hive schema metadata, leaving the physical data files intact in HDFS.

Q3. Explain the difference between Partitioning and Bucketing in Hive.

Partitioning organizes data into separate subdirectories based on column values (pruning queries). Bucketing splits data into a fixed number of files based on a hash of a column, which optimizes joins on high-cardinality columns.

Q4. Detail the write path of a record in Apache HBase.

A client write is first appended to the Write-Ahead Log (WAL) on disk for durability, then written to the in-memory **MemStore** in RAM. Once the write is in MemStore, the client receives an ACK.

Q5. When does HBase flush data, and what file format is generated?

When the in-memory MemStore reaches its capacity threshold (typically 128MB), HBase flushes its sorted key-value contents to HDFS, creating a new binary **HFile**.

Q6. Why is HBase called a Column-Family NoSQL database?

Frequently Asked / Viva Questions (contd.)

HBase groups columns together into **Column Families** (defined during table creation). Internally, columns belonging to the same column family are stored in the same physical HFile, optimizing access for selective queries.

Q7. What are the complex nested data types supported in Pig Latin?

Tuple (an ordered list of fields), **Bag** (an unordered collection of tuples), and **Map** (a set of key-value pairs).

Q8. Explain the role of ZooKeeper in an HBase cluster.

ZooKeeper coordinates Region assignment, detects RegionServer failures, maintains the root table location, and prevents active-standby conflicts by coordinating HMaster elections.

Q9. What are UDF, UDAF, and UDTF in Hive?

- **UDF:** User Defined Function (maps 1 row to 1 output).
- **UDAF:** User Defined Aggregate Function (maps multiple rows to 1 aggregated output).
- **UDTF:** User Defined Table Generating Function (maps 1 row to multiple output rows/tables).

Q10. Describe the execution flow of a Big SQL query.

Big SQL bypasses MapReduce. The Big SQL coordinator compiles SQL queries, optimizes execution plans, and uses an MPP engine to distribute query processing fragments directly to local nodes containing target HDFS blocks.

Comparison Snapshot

Traditional RDBMS vs. NoSQL HBase:

Feature	Traditional RDBMS	NoSQL HBase
Data Layout	Row-oriented (stores entire row together)	Column-family oriented (stores family columns together)
Schema Flexibility	Schema-on-write (rigid definitions)	Schema-less (columns can be added dynamically)
Transaction Support	Full ACID transactions across multiple tables	ACID transactions supported at Row-level only
Scaling Pattern	Vertical (bigger machines)	Horizontal (adding RegionServers in parallel)
Join Support	Rich declarative SQL joins	No native join support (requires MapReduce/Spark)
Null Value Storage	Consumes space for NULL definitions	Zero storage overhead for empty cells (sparse tables)